Solidez Mecanizada de un Verificador AIR/FRI Vinculado a Nivel de Bytes en el ROM Clásico Finito: De las Pruebas en Lean 4 a una Implementación Endurecida en Rust

Juan Diego Susunaga Velasquez
Escuela de Ingeniería, Ciencia y Tecnología
Universidad del Rosario
Bogotá, Colombia
juand.susunaga@urosario.edu.co

Pedro Mario Wightman Rojas
Escuela de Ingeniería, Ciencia y Tecnología
Universidad del Rosario
Bogotá, Colombia
pedro.wightman@urosario.edu.co

***Resumen*—Las Pruebas Interactivas de Oráculo de Proximidad (IOPPs) como FRI y las Representaciones Intermedias Algebraicas (AIR) suelen analizarse en modelos algebraicos idealizados donde los compromisos, los desafíos Fiat–Shamir y la selección de consultas actúan como pasos independientes. En contraste, los verificadores reales leen secuencias crudas de bytes, comparten un estado hash con memoria entre fases, ejecutan bucles acotados de deduplicación y verifican nonces de prueba de trabajo (*grinding*). Presentamos una mecanización libre de axiomas ad-hoc en Lean 4 (2,709 declaraciones verificadas en 347 submódulos) de la solidez (*soundness*) de un verificador AIR/FRI vinculado a nivel de bytes en el Modelo de Oráculo Aleatorio (ROM) clásico finito sobre el cuerpo primo Goldilocks ($p = 2^{64} - 2^{32} + 1$). Nuestra formalización modela el lector canónico de bytes, una tabla ROM compartida bajo Q consultas adaptativas, la congelación causal de oráculos y un selector acotado que extrae 40 parejas distintas en a lo sumo $R = 512$ intentos. Refutamos formalmente la premisa de "frescura universal" en cadenas con múltiples raíces, acotamos el error de acoplamiento $\delta(B)$ y componemos $g \in \{12, 20\}$ bits de trabajo sobre a lo sumo $Q + 1$ parejas estado/nonce elegibles (por ejemplo, $\Pr[\mathsf{AceptaFalsa}_{12}] < 0.004757335454$ para $T = 4096$, $g = 20$ y $Q = 2^{20}$). Finalmente, describimos cómo la formalización guió el endurecimiento del verificador en Rust, eliminando vulnerabilidades de aceptación en el verificador legado a distancia relativa de Hamming $7/8$.**

***Index Terms*—Pruebas Interactivas de Oráculo, FRI, STARK, Modelo de Oráculo Aleatorio, Lean 4, Verificación Formal, Rust.**

I. Introducción

Los argumentos sucintos basados en Representaciones Intermedias Algebraicas (AIR) y Pruebas Interactivas de Oráculo de Proximidad de Reed–Solomon Rápidas (FRI) [4], [5] constituyen la base de los sistemas modernos de cálculo verificable y rollups de validez de Capa 2 (L2).

Aunque la solidez teórica de FRI y DEEP-ALI ha sido ampliamente estudiada en la literatura criptográfica [6], [12], persiste una brecha importante entre las demostraciones idealizadas en papel y los verificadores reales que procesan bytes:

1) **Lectura a Nivel de Bytes y Validación Previa al Hash:** Los análisis en papel presuponen objetos algebraicos bien tipados. Los verificadores reales leen arreglos de bytes no confiables donde residuos no canónicos ($x \geq p$), bytes sobrantes o capacidades de vectores controladas por el atacante pueden provocar agotamiento de memoria o maleabilidad de la transcripción antes de invocar el hash.
2) **ROM Compartido con Memoria frente a Frescura Idealizada:** La transformación de Fiat–Shamir [7], [10] suele analizarse tratando cada extracción como una muestra uniforme independiente. En una transcripción concreta que absorbe raíces de Merkle secuenciales y nonces sobre una misma función hash, un adversario adaptativo puede preconsultar, repetir o intercalar llamadas al oráculo. Suponer "frescura universal" en cadenas de absorción de raíces es matemáticamente imposible desde que se absorben dos o más raíces.
3) **Bucles Acotados de Deduplicación y Muestra Compartida:** Para muestrear $q = 40$ parejas distintas sobre $M = 2^{k+2}$ parejas plegadas sin reemplazo, los verificadores concretos extraen índices iterativamente descartando duplicados hasta un corte fijo ($R = 512$ extracciones). Además, la misma muestra de 40 parejas se comparte simultáneamente entre las aperturas AIR y dos recorridos FRI vinculados (traza y composición), invalidando cualquier cota producto ingenua.
4) **Composición Adaptativa del Trabajo (*Grinding*):** La prueba de trabajo de g bits suele incorporarse multiplicando el error de solidez por $2^{-g}Q$. Justificar formalmente este factor exige demostrar

que toda ejecución aceptada proviene de una primera exposición estado/nonce en la historia del ROM, que existen a lo sumo $Q+1$ parejas elegibles considerando reintentos, y que la masa 2^{-g} se compone localmente con el selector acoplado antes de observar las respuestas de consulta.

En este artículo, desarrollado en el marco del proyecto **NEXUS**, cerramos esta brecha mecanizando de extremo a extremo el verificador AIR/FRI vinculado a nivel de bytes en **Lean 4** (v4.35.0 con `mathlib4`) y adaptando nuestra implementación en **Rust** al contrato formal resultante.

I-A. Contribuciones

- **Formalización ROM a Nivel de Bytes en Lean 4:** Formalizamos el lector canónico de bytes, el validador de formas, el canal Fiat–Shamir, el verificador de seis aperturas AIR y el doble verificador FRI vinculado sobre el cuerpo Goldilocks $\mathbb{F}_p$. La mecanización suma **2,709 declaraciones en 347 submódulos**, auditadas sin `sorryAx`.
- **Selector Acoplado con Memoria y Congelación Causal:** Demostramos que un adversario adaptativo (Q consultas) frente a un ROM con memoria coincide con el muestreo uniforme perezoso sobre prefijos efectivos finitos. Establecemos la congelación causal de los oráculos antes de su primera exposición y acotamos la distancia entre el selector acotado ($R = 512$) y una extracción hipergeométrica ideal mediante $\delta(B)$.
- **Cotas de Solidez Concretas Certificadas por Máquina:** Derivamos cotas racionales cerradas tanto para un perfil fijo ($k \in \{4, \ldots, 12\}$, $T = 2^k$ filas) como para perfiles elegidos adaptativamente, cobrando las colisiones del registro hash (Bad) y las excepciones algebraicas AIR/FRI una sola vez de forma global y ponderando el riesgo del selector por 2^{-g} sobre a lo sumo $Q + 1$ parejas.
- **Codiseño y Endurecimiento del Verificador en Rust:** Documentamos cómo la formalización guió el rediseño de `nexus-zk::stark::linked` y `nexus-active::linked_verifier`, eliminando fallos de solidez del verificador heredado (demostrados mediante tres contraejemplos a distancia relativa 7/8 aceptados por el verificador antiguo) e imponiendo contexto externo.
- **Delimitación Epistémica Explícita:** Separamos con rigor lo que demuestran los teoremas en Lean 4 en el ROM clásico finito de lo que permanece fuera del modelo mecanizado (criptoanálisis de SHA3-256 concreto, refinamiento universal del binario de máquina y transferencia al modelo cuántico [QROM]).

II. Especificación del Protocolo y Modelo de Amenazas

II-A. Dominio Algebraico y Estructura AIR/FRI Vinculada

El verificador opera sobre el cuerpo primo Goldilocks $\mathbb{F}_p$ con $p = 2^{64} - 2^{32} + 1 = 18{,}446{,}744{,}069{,}414{,}584{,}321$, y su extensión cuadrática $\mathbb{F}_{p^2} = \mathbb{F}_p[u]/(u^2 - 7)$ para los retos de plegado. Un perfil de ejecución está parametrizado por $k \in \{4, 5, \ldots, 12\}$, correspondiente a una traza de $T = 2^k \in [16, 4096]$ filas.

Con un factor de expansión (*blowup*) de 8, el dominio de evaluación LDE tiene tamaño $N = 8T = 2^{k+3}$ sobre un coset multiplicativo $D = 7 \cdot \langle \omega_N \rangle \subset \mathbb{F}_p^\times$, donde $\omega_N^8 = g_T$ genera el subgrupo de la traza de orden T y $\omega_N^{N/2} = -1$.

Se comprobó exhaustivamente sobre los 65,408 puntos de los nueve dominios admitidos la ausencia de polos ($x \notin \{0, 1, g_T^{T-1}\}$ y $x^T \neq 1$). Los puntos opuestos $(x, -x)$ se agrupan en $M = N/2 = 2^{k+2}$ parejas plegadas con profundidad de árbol $d = k + 3$.

A diferencia de las pruebas FRI aisladas, nuestro verificador comprueba una relación **doble FRI vinculada**:

1) **Compromiso de Traza y Aperturas AIR:** Para cada índice de pareja muestreado $i \in \{0, \ldots, M - 1\}$, el verificador autentica seis posiciones en $[0, N)$ contra la raíz de Merkle compartida de la traza (las filas actuales y desplazadas tanto en x como en $-x$, comprobadas en las 196,224 configuraciones de índices posibles).
2) **Enlace del Polinomio de Composición:** Usando los retos Fiat–Shamir extraídos tras absorber la raíz de la traza, el verificador evalúa las restricciones de transición y de frontera en x y $-x$ y comprueba que coincidan con la capa inicial del compromiso de composición.
3) **Doble Plegado FRI:** El verificador ejecuta simultáneamente $m_{\text{traza}} = k$ rondas de plegado para la traza y $m_{\text{comp}} = k+1$ rondas de plegado para el polinomio de composición sobre $\mathbb{F}_{p^2}$, reutilizando exactamente las mismas 40 parejas de índices muestreadas.

II-B. Gramática de Bytes del Certificado y Presupuesto de Llamadas

Un certificado serializado (S, π) es una secuencia cruda de octetos (`List Octeto` en Lean, `&[u8]` en Rust) acotada por un límite estricto de 2 MiB (2,097,152 bytes). Para el perfil k ($d = k+3$), la longitud en bytes de cada compromiso es

$$\mathsf{Bytes}_{\text{com}}(m) = 24 + 32m, \tag{1}$$

el tamaño del recorrido de plegado FRI por fase es

$$\mathsf{Bytes}_{\text{fri}}(m) = 8 + \sum_{j=0}^{m-1} \big(48 + 64(d - j)\big), \tag{2}$$

Tabla I
Tamaños Exactos de Certificado y Presupuesto de Hashes del Verificador ($R = 512$)

Perf. k	**Filas** T	**Dom.** N	**Bytes Cert.**	**Hashes Máx.** $V_{\text{máx}}$
4	16	128	196,574	6,943
5	32	256	249,118	8,627
6	64	512	306,782	10,471
7	128	1,024	369,566	12,475
8	256	2,048	437,470	14,639
9	512	4,096	510,494	16,963
10	1,024	8,192	588,638	19,447
11	2,048	16,384	671,902	22,091
12	4,096	32,768	760,286	24,895

el tamaño de cada apertura de consulta es

$$\begin{aligned}\mathsf{Bytes}_{\text{consulta}}(k) = 8 + 6(16 + 32d) \\ + \mathsf{Bytes}_{\text{fri}}(k) + \mathsf{Bytes}_{\text{fri}}(k+1),\end{aligned} \tag{3}$$

y la longitud total en bytes del certificado es

$$\begin{aligned}\mathsf{Bytes}(k) = 86 + \mathsf{Bytes}_{\text{com}}(k) + \mathsf{Bytes}_{\text{com}}(k+1) \\ + 40 \cdot \mathsf{Bytes}_{\text{consulta}}(k).\end{aligned} \tag{4}$$

Como se establece en (1)–(4), el tamaño exacto de cualquier certificado válido queda determinado de forma única por k. La Tabla I resume los tamaños exactos obtenidos mediante (4) y el presupuesto máximo de llamadas al oráculo por parte del verificador en los nueve perfiles.

Incluso en el perfil más grande ($k = 12$, $T = 4096$), el certificado ocupa 760,286 bytes (bajo el límite de 2 MiB) y el verificador realiza a lo sumo $V_{\text{máx}}(12) = 24{,}895$ llamadas al oráculo (incluyendo hasta $R = 512$ extracciones del selector), incluso cuando rechaza.

II-C. ROM Clásico Finito y Adversario Adaptativo

Modelamos la función hash como un único Oráculo Aleatorio clásico compartido $H : \mathcal{U} \to \{0,1\}^{256}$ (`Digest`) con memoria persistente durante todo el experimento:

- Cada entrada nueva $x \in \mathcal{U}$ devuelve una respuesta uniforme e independiente de 256 bits en su *primera* consulta; cualquier llamada posterior con la misma entrada x (del adversario o del verificador) repite exactamente la respuesta registrada.
- El adversario adaptativo ($\mathsf{proponer}, \mathsf{atacar}, \mathsf{actualizar}$), parametrizado por un tipo de estado arbitrario A, realiza a lo sumo Q consultas al oráculo durante la preparación, búsqueda de nonce e intercalaciones.
- Sumando el costo máximo del verificador (24,895 llamadas para $R = 512$), el número total de entradas distintas en la tabla compartida está acotado por el presupuesto común $B = Q + 24{,}895$, con una cota auxiliar de marcas de registro L.
- El trabajo exige $g \in \{12, 20\}$ bits iniciales nulos en el resumen del trabajo absorbido con un nonce de 64 bits antes de extraer los índices de consulta.

III. Solidez Mecanizada en Lean 4

Nuestra formalización en Lean 4 se estructura en 347 submódulos bajo el espacio de nombres `Nexus`, culminando en `Nexus.Verificador.CotaTrabajoFinal` y `Nexus.Verificador.CotaTrabajoNumerica`. En lugar de axiomatizar estados intermedios ideales, demostramos la cota directamente desde el programa verificador de bytes.

III-A. Refutación de la Frescura Universal Ingenua y Congelación Causal

Un error frecuente al formalizar Fiat–Shamir en protocolos de múltiples rondas (como FRI, que absorbe hasta $m \leq 30$ raíces secuenciales) consiste en asumir como hipótesis que cada absorción consulta un punto fresco con probabilidad 1. En Lean 4 demostramos que dicha premisa de *frescura universal* es matemáticamente imposible desde dos raíces, ya que un adversario adaptativo puede reutilizar deliberadamente raíces o prefijos previos.

En su lugar, construimos un acoplamiento con memoria sobre el árbol finito de ejecución ROM (`LeyArbol` y `ObservacionesPrefijo`):

1) **Integridad del Registro Hash (SinBad):** Definimos un detector computable sobre el registro de ejecución que captura cualquier colisión, preimagen anticipada o ciclo de prefijos en la tabla. En `CotaSinBad`, probamos que la probabilidad de violar SinBad bajo el presupuesto $B = Q + 24{,}895$ y parámetro auxiliar L satisface
$$\Pr[\neg\mathsf{SinBad}] \leq \min\left\{1, \frac{B(49B + L)}{2^{256}}\right\}. \tag{5}$$
2) **Congelación Causal de Compromisos (`PrefijoAIR`, `RondasCausales`):** Bajo la condición SinBad de (5), antes de la primera respuesta de reto (E) de cada capa FRI o combinación AIR, la raíz de Merkle correspondiente se recupera de forma única por búsqueda inversa en la tabla previa. Así, el vector de evaluaciones comprometido queda causalmente congelado en el pasado antes de muestrear el reto.
3) **Unión Telescópica de Excepciones Algebraicas y Transporte al Cuerpo (`CotaAdaptativaFinal`):** Para todo $k \in \{4, \ldots, 12\}$, ambos recorridos FRI y la combinación AIR, el conjunto excepcional de retos en $\mathbb{F}_{p^2}$ previo a cada extracción E tiene cardinal a lo sumo 130,709 (130,708 de la suma telescópica FRI más 1 del conjunto crítico de cancelación AIR). Incorporando el sesgo de reducción modular al proyectar 256 bits sobre $\mathbb{F}_p^2$, cada consulta fresca de reto tiene una tasa de excepción acotada por
$$\rho = \frac{130{,}709}{p^2} + \frac{2{,}524{,}360}{10^{35}}, \tag{6}$$
lo que arroja una cota global ρB sobre toda la historia.

III-B. Selector Acotado Acoplado ($R = 512$) y Composición con Trabajo

Cuando el oráculo comprometido está lejos del código (menos de $\lfloor 7{\cdot}2^{k+2}/8 \rfloor$ de las $M = 2^{k+2}$ parejas son "buenas" conjuntamente para AIR y ambos FRI), el verificador solo acepta si las 40 parejas distintas extraídas caen dentro del subconjunto bueno $G_T \cap G_C$, o si el selector se desvía del muestreo sin reemplazo.

En `SelectorAcoplado` y `SelectorAdaptativo`, acoplamos el selector real con memoria (hasta $R = 512$ extracciones para reunir 40 índices distintos) con una cinta ideal independiente. Siguiendo exactamente el orden de la sección 6.7.9 de la tesis NEXUS, para el primo Goldilocks $p = 2^{64} - 2^{32} + 1$ y el presupuesto compartido $B = Q + 24{,}895$, además de ρ en (6), el error de acoplamiento con memoria es

$$\delta(B) = \min\left\{1, \frac{512(B + 515)}{2^{256}}\right\}, \tag{7}$$

la probabilidad hipergeométrica exacta de omisión de la muestra de 40 parejas es

$$h_k = \frac{\binom{\lfloor 7{\cdot}2^{k+2}/8 \rfloor}{40}}{\binom{2^{k+2}}{40}}, \tag{8}$$

y el riesgo combinado del selector es

$$\sigma_k(B) = h_k + 2^{-300} + \delta(B). \tag{9}$$

En (9), h_k definido en (8) expresa la cota de omisión de la muestra, 2^{-300} acota la cola de abandono tras 512 extracciones, y $\delta(B)$ definido en (7) cubre el error de acoplamiento con memoria sobre $B + 2$ pasos efectivos.

Lema 1 (Riesgo del Selector por Primera Exposición)**.** *Condicionado a cualquier pasado fijo de a lo sumo B entradas ROM y una primera dirección estado/nonce fresca, la probabilidad de que el selector acotado a $R = 512$ extraiga 40 índices distintos contenidos en un conjunto de cardinal $\leq \lfloor 7 \cdot 2^{k+2}/8 \rfloor$ está acotada por $\sigma_k(B)$ según (9).*

En `CotaInicios` y `TrabajoSelectorLocal` demostramos dos propiedades estructurales clave:

1) **A lo sumo $Q + 1$ Parejas Estado/Nonce Elegibles:** Cada inicio del selector se inyecta en una absorción estado/nonce distinta; en toda la ejecución (incluidos rechazos, repeticiones e intercalaciones) existen a lo sumo $Q + 1$ parejas.
2) **Composición Local del Trabajo (2^{-g}):** Para $g \in \{12, 20\}$, el control de trabajo tiene masa 2^{-g} en su primera consulta y se compone con el riesgo del selector antes de observar las respuestas, dando $2^{-g}\sigma_k(B)$ por pareja elegible.

III-C. Teoremas Principales de Solidez Mecanizados

La combinación de (5)–(9) produce los teoremas centrales de `Nexus.Verificador.CotaTrabajoFinal`, enunciados a continuación y acompañados por su formalización exacta en el Listado 1.

```
@[expose] noncomputable def selector (k B : ℕ) : ℚ :=
  (Nat.choose (CardinalVista.umbral (k + 2)) 40 : ℚ) /
    Nat.choose (2 ^ (k + 2)) 40 +
    1 / (2 : ℚ) ^ 300 +
    AcoplamientoCadena.error Digest 512 (B + 2)

@[expose] noncomputable def cota512 (L bits k Q : ℕ) : ℚ :=
  let B := Q + 24895
  min 1 (((B * (49 * B + L) : ℕ) : ℚ) / (2 : ℚ) ^ 256) +
    CotaAdaptativaFinal.tasa * B +
    (1 / (2 : ℚ) ^ bits) * selector k B * (Q + 1)

theorem cota_512 {A : Type} (L bits k Q : ℕ)
    (proponer : Programa (List Octeto × A))
    (atacar : A → GramaticaTG2.Peticion → Programa A)
    (actualizar : A → GramaticaTG2.Peticion → Digest → A)
    (defecto : Digest)
    (hbits : bits = 12 ∨ bits = 20)
    (hk0 : 4 ≤ k) (hk1 : k ≤ 12)
    (hQ : ∀ H, Adversario.adversarias
      (oraculo (Adversario.ejecutar bits 512
        proponer atacar actualizar) defecto H)
      bits 512 proponer atacar actualizar ≤ Q) :
    let p := Adversario.ejecutar bits 512
      proponer atacar actualizar
    medida (fun H : universo p → Digest =>
      PoliticasFalsas.AceptaFalsa
        (oraculo p defecto H) bits 512 k
        proponer atacar actualizar) ≤
      min 1 (cota512 L bits k Q)
```

Listado 1. Cota principal de solidez mecanizada en Lean 4 (`CotaTrabajoFinal.lean`).

Teorema 1 (Solidez a Nivel de Bytes para Perfil Fijo)**.** *Para todo adversario adaptativo con presupuesto de consultas Q, perfil $k \in \{4, \ldots, 12\}$, trabajo $g \in \{12, 20\}$ y parámetro auxiliar L (con $B = Q + 24{,}895$), la probabilidad en el ROM clásico finito de que el verificador de bytes acepte un certificado con enunciado falso satisface*

$$\Pr[\mathsf{AceptaFalsa}_k] \leq \min\Bigg\{1,\ \min\left\{1, \frac{B(49B + L)}{2^{256}}\right\} + \rho B + (Q + 1)2^{-g}\sigma_k(B)\Bigg\}. \tag{10}$$

Teorema 2 (Solidez Adaptativa Multi-Perfil)**.** *Si el adversario elige adaptativamente el perfil de salida $k \in \{4, \ldots, 12\}$, la probabilidad de aceptación falsa sobre todos los perfiles satisface*

$$\Pr[\mathsf{AceptaFalsa}] \leq \min\Bigg\{1,\ \min\left\{1, \frac{B(49B + L)}{2^{256}}\right\} + \rho B + (Q + 1)2^{-g}\sum_{j=4}^{12}\sigma_j(B)\Bigg\}. \tag{11}$$

En (11), solo se suma por perfil el riesgo del selector; la excepción del registro hash y las excepciones AIR/FRI (ρB) se incluyen una sola vez.

En `CotaTrabajoNumerica.lean`, instanciamos (10) y (11) en aritmética racional exacta (`norm_num`):

- Para $L = 10^6$, $k = 12$ ($T = 4096$ filas), $g = 20$ bits y $Q = 2^{20}$ (`aceptacion_4096_release`), la evaluación

Tabla II
Correspondencia entre los Módulos Formales en Lean 4 y la Implementación Endurecida en Rust

Capa del Verificador	Módulos en Lean 4	Módulos en Rust	Invariante de Seguridad Aplicado
1. Lector Canónico y Rechazo Previo al Hash	`Nexus.Verificador.{Certificado, Forma, Completo}`	`nexus-zk::stark::{linked_codec, linked}`	Lectura little-endian explícita, residuos canónicos ($x < p$), EOF estricto y capacidades derivadas del perfil antes de asignar vectores.
2. Canal Fiat–Shamir y Perfil de Trabajo	`Nexus.Canal.{GramaticaTG2, ProgramaTG2, DesafiosPrograma}`	`nexus-zk::stark::{linked, channel}`	Dominios, etiquetas, orden de desafíos y exigencia de perfil externo ($g \in \{12, 20\}$); el certificado no puede rebajar g.
3. Selector Ordenado Acotado ($R = 512$)	`Nexus.Canal.SelectorOrdenado`	`nexus-zk::stark::linked`	Extrae exactamente 40 índices distintos por orden de primera aparición en ≤ 512 extracciones; rechaza si requeriría la 513.
4. Enlace AIR y Doble FRI	`Nexus.Verificador.Consulta`, `Nexus.FRI.{ProgramaVerificacion, Recorrido, Cosets}`	`nexus-zk::stark::{linked, fri, merkle, field}`	Autentica 6 posiciones contra raíz compartida, evalúa composición en x y $-x$, y pliega traza (k) y composición ($k+1$) sobre $\mathbb{F}_{p^2}$.
5. Contexto Local y Registro de Raíces	*Capa de aplicación externa al teorema AIR*	`nexus-active::{linked_verifier, verifier}`	`NitroVerifier::verify_expected` exige contexto local previo al registro, impide sobrescribir alturas y separa el modo optimista.

de (10) arroja

$$\Pr[\mathsf{AceptaFalsa}_{12}] < \frac{4{,}757{,}335{,}454}{10^{12}} = 0.004757335454. \tag{12}$$

- Sobre los nueve perfiles adaptativamente, con $L = 10^6$, $g = 20$ bits y $Q = 10^6$ (`aceptacion_release`), aplicando $h_k \leq 1/200$ en (11) se obtiene

$$\Pr[\mathsf{AceptaFalsa}] < \frac{1}{20} = 0.05. \tag{13}$$

- Para el perfil de desarrollo $g = 12$ bits y $Q = 10^3$ (`ejemplo_debug`), (11) entrega

$$\Pr[\mathsf{AceptaFalsa}] < \frac{1}{80} = 0.0125. \tag{14}$$

IV. Del Modelo en Lean 4 al Ejecutable en Rust

Una lección central de este trabajo es que la mecanización en Lean 4 expuso discrepancias concretas en nuestro prototipo inicial en Rust (`nexus-zk`) que las pruebas unitarias habían pasado por alto.

IV-A. Refutación del Verificador Legado con Contraejemplos a Distancia 7/8

Antes de la formalización, el verificador STARK experimental heredado en `nexus-zk` validaba un compromiso polinómico aislado sin vincular el polinomio de composición en un doble recorrido FRI.

Durante nuestra auditoría de transferencia de distancia (S2), construimos tres trazas adversarias explícitas a distancia relativa de Hamming 7/8 del código de grado $< T$ (para $T = 16$ filas) con enunciados públicos válidos que eran *aceptadas* por el verificador heredado. Para eliminar esta vulnerabilidad, reemplazamos la ruta canónica por el contrato vinculado (`nexus-zk::stark::linked`).

IV-B. Alineación Estructural entre Lean 4 y Rust

La Tabla II resume las cinco capas sincronizadas entre la especificación en Lean 4 y los crates endurecidos en Rust (`nexus-zk` y `nexus-active`).

IV-C. Endurecimiento de la Capa de Aplicación (`ActiveChain`)

En `nexus-active`, conectamos `ActiveChain` con `NitroVerifier::verify_expected` bajo tres invariantes:

- **Contexto de Instantánea Privada:** `ActiveChain` conserva instantáneas privadas de los lotes construidos localmente; el perfil de filas y el trabajo g se fijan por configuración del nodo y no desde el sobre externo.
- **Verificación Previa al Registro:** La ruta de validez e híbrida ejecuta `verify_expected` antes de registrar cualquier envío o raíz, impidiendo duplicados concurrentes mediante exclusión mutua.
- **Sin Degradación Silenciosa:** Las llamadas sin contexto esperado rechazan incondicionalmente, aislando el modo optimista de las raíces verificadas.

En la verificación consolidada del workspace pasan las **223 pruebas globales** (y **436 pruebas** en la suite extendida con diario BFT v2), incluidos los 14 certificados de regresión (8 honestos aceptados y 6 lejanos rechazados).

V. Discusión: Límites Epistémicos y Alcance

1) **ROM Clásico Finito frente a SHA3-256 Concreto:** Las cotas demostradas en (10) y (11) rigen en el modelo ROM ideal con memoria; el ejecutable Rust instancia el canal con `SHA3-256`. La prueba en el modelo ideal no demuestra automáticamente la misma garantía para la función concreta SHA3-256.

2) **Régimen de Parámetros frente a 100 Bits de Seguridad:** La ecuación (12) describe un experimento matemático con 40 parejas, $g = 20$ bits y $Q = 2^{20}$; no acredita por sí sola 100 bits de seguridad ni es una tasa de ataque por transacción.
3) **Recurrencia AIR Escalar frente a Transacciones L2 Generales:** El AIR formalizado comprueba una recurrencia escalar sobre $\mathbb{F}_p$ y rechaza lotes con raíces SHA3 generales fuera del modelo, sin recortar hashes ni fabricar pruebas sintéticas para transacciones completas.
4) **Ausencia de Refinamiento Universal de Máquina y QROM:** Aunque auditamos estáticamente la correspondencia entre las 88 fuentes Rust y 354 fuentes Lean, la equivalencia universal de máquina y la extensión completa al modelo cuántico QROM [8], [14] permanecen como trabajo futuro.

VI. Trabajo Relacionado y Conclusión

Esfuerzos recientes como **ArkLib** [2] en Lean 4 y formalizaciones en Coq, EasyCrypt e Isabelle/HOL [1], [3], [9], [11], [13], [15] se enfocan principalmente en reducciones algebraicas modulares para IOPs, esquemas de compromiso polinómico o aritmética de cuerpo de bajo nivel. Nuestro trabajo los complementa al mecanizar de extremo a extremo el *verificador concreto de bytes Fiat–Shamir* (2,709 declaraciones libres de axiomas ad-hoc en 347 submódulos), incluyendo la deserialización binaria determinista, el registro ROM con detección de colisiones y ciclos (SinBad en (5)), el selector acotado a $R = 512$ extracciones en (9) y las cotas racionales certificadas en (10)–(14) mediante `norm_num`.

Al acoplar el lector de bytes, la memoria del ROM y el trabajo de g bits sobre $Q + 1$ parejas elegibles, nuestra formalización elimina las hipótesis idealizadas de independencia y guió directamente el endurecimiento del verificador en Rust dentro del proyecto NEXUS. Como trabajo futuro se abordarán las cotas de superposición en QROM para FRI vinculado y la validación mecanizada de traducción entre Rust y Lean 4.

Referencias

[1] J. B. Almeida, M. Barbosa, G. Barthe, A. Blot, B. Grégoire, V. Laporte, T. Oliveira, H. Pacheco, B. Schmidt, and P.-Y. Strub, "Jasmin: High-assurance and high-speed cryptography," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2017, pp. 1807–1823.

[2] ArkLib Contributors, "ArkLib: A formal library of interactive oracle proofs and arguments of knowledge in Lean 4," GitHub repository, 2024.

[3] G. Barthe, B. Grégoire, S. Heraud, and S. Zanella-Béguelin, "Computer-aided security proofs for the working cryptographer," in *Advances in Cryptology – CRYPTO 2011*, Springer, 2011, pp. 71–90.

[4] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Fast Reed–Solomon interactive oracle proofs of proximity," in *Proc. 45th Int. Colloq. Automata, Languages, and Programming (ICALP)*, 2018, pp. 14:1–14:17.

[5] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Scalable, transparent, and post-quantum secure computational integrity," *IACR Cryptology ePrint Archive*, Rep. 2018/046, 2018.

[6] E. Ben-Sasson, D. Carmon, Y. Ishai, S. Kopparty, and S. Saraf, "Proximity gaps for Reed–Solomon codes," in *Proc. IEEE 61st Annu. Symp. Found. Comput. Sci. (FOCS)*, 2020, pp. 900–909.

[7] E. Ben-Sasson, A. Chiesa, and N. Spooner, "Interactive oracle proofs," in *Theory of Cryptography Conf. (TCC 2016-B)*, Springer, 2016, pp. 31–60.

[8] A. Chiesa, P. Manohar, and N. Spooner, "Succinct arguments in the quantum random oracle model," in *Theory of Cryptography Conf. (TCC)*, Springer, 2019, pp. 1–29.

[9] A. Erbsen, J. Philipoom, J. Gross, R. Sloan, and A. Chlipala, "Simple high-level code for cryptographic arithmetic – with proofs, without compromises," in *Proc. IEEE Symp. Security and Privacy (SP)*, 2019, pp. 1202–1219.

[10] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Advances in Cryptology – CRYPTO '86*, Springer, 1986, pp. 186–194.

[11] L. Goldberg, S. Papini, and M. Riabzev, "Cairo – a Turing-complete STARK-friendly CPU architecture," *IACR Cryptology ePrint Archive*, Rep. 2021/1063, 2021.

[12] U. Haböck, "A summary on the FRI low degree test," *IACR Cryptology ePrint Archive*, Rep. 2022/1216, 2022.

[13] A. Lochbihler, S. R. Sefidgar, D. Basin, and U. Maurer, "Formalizing constructive cryptography using CryptHOL," in *Proc. IEEE 32nd Comput. Security Found. Symp. (CSF)*, 2019, pp. 152–166.

[14] M. Zhandry, "How to record quantum queries, and applications to quantum indifferentiability," in *Advances in Cryptology – CRYPTO 2019*, Springer, 2019, pp. 239–268.

[15] J.-K. Zinzindohoué, K. Bhargavan, J. Protzenko, and B. Beurdouche, "HACL*: A verified modern cryptographic library," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2017, pp. 1789–1806.